

ParcelPilot Troubleshooting Runbook Collection

Runbooks for login, orders, tracking, documents, notifications, reporting, and unclear incidents.

ParcelPilot operations reference

Contents

1. Login failure
2. Order not found
3. Order page returns 500
4. Migration mismatch
5. Tracking partial
6. Tracking stale
7. Tracking timeout
8. Document failure
9. Notification delay
10. Report timeout
11. Generic API error
12. Insufficient evidence

1. Login failure

This runbook is for login failure problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Check whether one or many users are affected.
- Inspect auth service status.
- Check AUTH-401, AUTH-423, and AUTH-503 around the failure time.
- For one user, inspect account-lock state.
- For many users, look for service unavailability.

Expected evidence

Useful evidence normally comes from the auth path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Operator notes

Runbooks are a starting order, not a substitute for the evidence in front of you. If an early check already shows a clear and specific failure, record it before continuing. If a check contradicts the expected pattern, adjust the path rather than forcing the incident through every step.

When several services show warnings, prioritize the service actually used by the failing request. A tracking warning should not outrank a repeated orders database error when the reported problem is that basic order details return 500.

Handoff checklist

- What failed and what still worked.
- Which checks were performed.
- Which evidence supports the current conclusion.
- Which question remains unanswered.
- What should be tested next.

If no supplied evidence supports a root cause, the runbook should end with the next useful check rather than a guessed diagnosis.

2. Order not found

This runbook is for order not found problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Confirm the order identifier and environment.
- Try a known-good order.
- Check for ORD-404.
- Do not treat one missing order as a service outage.
- Escalate only if known-good orders also fail.

Expected evidence

Useful evidence normally comes from the orders path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Operator notes

Runbooks are a starting order, not a substitute for the evidence in front of you. If an early check already shows a clear and specific failure, record it before continuing. If a check contradicts the expected pattern, adjust the path rather than forcing the incident through every step.

When several services show warnings, prioritize the service actually used by the failing request. A tracking warning should not outrank a repeated orders database error when the reported problem is that basic order details return 500.

Handoff checklist

- What failed and what still worked.
- Which checks were performed.
- Which evidence supports the current conclusion.
- Which question remains unanswered.
- What should be tested next.

If no supplied evidence supports a root cause, the runbook should end with the next useful check rather than a guessed diagnosis.

3. Order page returns 500

This runbook is for order page returns 500 problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Test more than one order ID.
- Check orders service status.
- Inspect ORD-500 and ORD-409.
- Check ORDERS_DB_PATH presence and readability.
- Correlate API-500 with the downstream orders request.

Expected evidence

Useful evidence normally comes from the orders path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Operator notes

Runbooks are a starting order, not a substitute for the evidence in front of you. If an early check already shows a clear and specific failure, record it before continuing. If a check contradicts the expected pattern, adjust the path rather than forcing the incident through every step.

When several services show warnings, prioritize the service actually used by the failing request. A tracking warning should not outrank a repeated orders database error when the reported problem is that basic order details return 500.

Handoff checklist

- What failed and what still worked.
- Which checks were performed.
- Which evidence supports the current conclusion.
- Which question remains unanswered.
- What should be tested next.

If no supplied evidence supports a root cause, the runbook should end with the next useful check rather than a guessed diagnosis.

4. Migration mismatch

This runbook is for migration mismatch problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Look for ORD-409.
- Compare expected and detected schema versions.
- Confirm the running application build.
- Do not delete the database as the first response.
- Use the documented training migration path.

Expected evidence

Useful evidence normally comes from the orders path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Operator notes

Runbooks are a starting order, not a substitute for the evidence in front of you. If an early check already shows a clear and specific failure, record it before continuing. If a check contradicts the expected pattern, adjust the path rather than forcing the incident through every step.

When several services show warnings, prioritize the service actually used by the failing request. A tracking warning should not outrank a repeated orders database error when the reported problem is that basic order details return 500.

Handoff checklist

- What failed and what still worked.
- Which checks were performed.
- Which evidence supports the current conclusion.
- Which question remains unanswered.
- What should be tested next.

If no supplied evidence supports a root cause, the runbook should end with the next useful check rather than a guessed diagnosis.

5. Tracking partial

This runbook is for tracking partial problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Confirm order details still load.
- Check TRK-206.
- Inspect the carrier snapshot for the order.
- Return available events with a partial note.
- Check whether a fresh snapshot later restores the missing event.

Expected evidence

Useful evidence normally comes from the tracking path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Normal operating pattern

Normal tracking reads a recent carrier snapshot and returns a sequence of shipment events. Some orders legitimately have fewer events than others, so the number of events alone is not an error.

Failure pattern

TRK-206 means the response is incomplete, TRK-409 means the snapshot is too old, TRK-422 means the source cannot be used safely, and TRK-504 means the carrier lookup exceeded its timeout.

Worked example

A snapshot updated eight minutes ago with two events for one order may be valid partial data. A snapshot updated two hours ago should be treated as stale even if the JSON structure is valid.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

6. Tracking stale

This runbook is for tracking stale problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Check TRK-409.
- Compare last_updated with incident time.
- Check refresh attempts and provider status.
- Do not label stale data as current.
- Keep existing historical events if they are valid.

Expected evidence

Useful evidence normally comes from the tracking path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Normal operating pattern

Normal tracking reads a recent carrier snapshot and returns a sequence of shipment events. Some orders legitimately have fewer events than others, so the number of events alone is not an error.

Failure pattern

TRK-206 means the response is incomplete, TRK-409 means the snapshot is too old, TRK-422 means the source cannot be used safely, and TRK-504 means the carrier lookup exceeded its timeout.

Worked example

A snapshot updated eight minutes ago with two events for one order may be valid partial data. A snapshot updated two hours ago should be treated as stale even if the JSON structure is valid.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

7. Tracking timeout

This runbook is for tracking timeout problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Count TRK-504 in the incident window.
- Compare tracking latency with order latency.
- Inspect provider state.
- Check several order IDs.
- Avoid increasing every timeout before locating the slow dependency.

Expected evidence

Useful evidence normally comes from the tracking path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Normal operating pattern

Normal tracking reads a recent carrier snapshot and returns a sequence of shipment events. Some orders legitimately have fewer events than others, so the number of events alone is not an error.

Failure pattern

TRK-206 means the response is incomplete, TRK-409 means the snapshot is too old, TRK-422 means the source cannot be used safely, and TRK-504 means the carrier lookup exceeded its timeout.

Worked example

A snapshot updated eight minutes ago with two events for one order may be valid partial data. A snapshot updated two hours ago should be treated as stale even if the JSON structure is valid.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

8. Document failure

This runbook is for document failure problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Identify the requested file.
- Check DOC-404, DOC-403, or DOC-500.
- Confirm order details still work.
- Check document-store path and file readability.
- Return a controlled not-found result for missing optional files.

Expected evidence

Useful evidence normally comes from the documents path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Normal operating pattern

Document routes first confirm the order and then look for the requested file under the configured store. Some orders do not have proof-of-delivery or customs documents, so a controlled not-found response can be normal.

Failure pattern

DOC-404, DOC-403, and DOC-500 should remain separate. A missing file, an unreadable file, and an invalid document root have different causes and different next checks.

Worked example

If label.pdf opens but proof.pdf returns DOC-404, do not change the global document-store path. Verify whether proof.pdf exists for that order.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

9. Notification delay

This runbook is for notification delay problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Check queue depth and oldest pending age.
- Look for NTF-429.
- Check provider state for NTF-503.
- Distinguish delayed from rejected messages.
- Avoid diagnosing order services for a queue-only symptom.

Expected evidence

Useful evidence normally comes from the notifications path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Operator notes

Runbooks are a starting order, not a substitute for the evidence in front of you. If an early check already shows a clear and specific failure, record it before continuing. If a check contradicts the expected pattern, adjust the path rather than forcing the incident through every step.

When several services show warnings, prioritize the service actually used by the failing request. A tracking warning should not outrank a repeated orders database error when the reported problem is that basic order details return 500.

Handoff checklist

- What failed and what still worked.
- Which checks were performed.
- Which evidence supports the current conclusion.
- Which question remains unanswered.
- What should be tested next.

If no supplied evidence supports a root cause, the runbook should end with the next useful check rather than a guessed diagnosis.

10. Report timeout

This runbook is for report timeout problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Check requested date range.
- Look for RPT-422 versus RPT-504.
- Check reporting service state.
- Compare small and large ranges.
- Do not change the orders database.

Expected evidence

Useful evidence normally comes from the reporting path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Normal operating pattern

Reporting reads a separate analytics database. Small date ranges normally complete faster than large ones, and the same report can return different row counts as the requested window changes.

Failure pattern

RPT-422 is a validation problem, RPT-500 is a query failure, and RPT-504 is a timeout after execution begins. A timeout should be investigated with date range, query type, and repeated duration.

Worked example

A 90-day report timing out while 7-day reports complete in 300 ms is not the same as every report failing immediately with RPT-500.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

11. Generic API error

This runbook is for generic api error problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Capture request_id.
- Find the downstream request with the same ID.
- Use the downstream code when available.
- Check the route path.
- Do not stop at API-500 if a specific service code exists.

Expected evidence

Useful evidence normally comes from the api path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Normal operating pattern

A normal API request has a request_id, a route, a downstream call when needed, and a final status. Basic health normally completes in tens of milliseconds, while an order route includes the time spent waiting for the orders service.

Failure pattern

A generic API-500 is most useful when correlated with a downstream code on the same request_id. If the downstream service returns a specific error, keep that code visible in the incident record instead of treating API-500 as the final diagnosis.

Worked example

For req-0048123, API-500 at 14:06:18 followed an ORD-500 from the orders service under the same request identifier. That pattern points to the orders path rather than the API process itself.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

12. Insufficient evidence

This runbook is for insufficient evidence problems. Keep the investigation scoped to the affected request and use the current incident time when filtering status and logs.

Checks

- Record what was checked.
- State which expected error was not found.
- Do not force the symptom into an old incident pattern.
- Ask for a narrower time or route when needed.
- Return the next useful check instead of a fake root cause.

Expected evidence

Useful evidence normally comes from the any path, the matching request ID, configuration used by that service, and the user flow that reproduces the problem.

Stop condition

Stop adding checks when the evidence supports a clear cause or when the available data no longer supports a stronger conclusion. If the latter happens, keep the incident investigating.

Verification

After a change, rerun the original request and one nearby case. Record whether the result matches expected behavior.

Operator notes

Runbooks are a starting order, not a substitute for the evidence in front of you. If an early check already shows a clear and specific failure, record it before continuing. If a check contradicts the expected pattern, adjust the path rather than forcing the incident through every step.

When several services show warnings, prioritize the service actually used by the failing request. A tracking warning should not outrank a repeated orders database error when the reported problem is that basic order details return 500.

Handoff checklist

- What failed and what still worked.
- Which checks were performed.
- Which evidence supports the current conclusion.
- Which question remains unanswered.
- What should be tested next.

If no supplied evidence supports a root cause, the runbook should end with the next useful check rather than a guessed diagnosis.